

AeghisDNS — Project Review

An independent assessment of the Capstone project: what works well, what to watch out for, and where it could go next.

Overview

AeghisDNS is an ambitious three-part security tool: a PySide6 desktop application, a Chrome browser extension, and a local Flask bridge between them. Together they let a user inspect, score, and intercept browsing activity — combining manual URL/domain/IP scanning, real-time packet monitoring, navigation logging, and (recently) a local LLM-driven AI Overview that explains scan results in plain English.

For a capstone-level project this is a lot — and most of it works. The architecture is clean, the feature set is genuinely useful, and the recent additions (model download flow, AI overview caching, public DNS resolver) show the system is being actively improved.

What it does well

Architecture

The split between **GUI** (src/gui), **business logic** (src/logic), the **standalone scoring engine** (scanner/), and the **packet subsystem** (sniffer_test/) is sensible and easy to follow. New features slot in cleanly because each layer has a clear responsibility.

Feature breadth

- Local heuristic scanner with explainable, weighted signals (WHOIS, DNS, TLS, HTTP).
- VirusTotal Deep Scan integration with rate-limit cooldown that survives restarts.
- Live packet sniffer (Scapy) feeding a rolling chart and threshold-based tray alerts.
- Chrome extension with four operational modes — None, Logging, Silent, and Safe — including a genuine pre-navigation interstitial flow.
- User-scoped white/black lists with smart domain normalization (alias matching, base-domain match).
- Five themes plus user account management (change username/password, delete account).
- AI Overview powered by a fully local Llama 3.2 3B model — no API key, no recurring cost, and a one-time download flow that handles missing-model gracefully.

Engineering decisions worth keeping

- Dual-write persistence: rich JSONL for cache and history, plus SQLite for the address table. Pragmatic and gives you both flexibility and queryability.

- QThread used consistently for everything that could block the UI (scans, VT calls, LLM, downloads).
- Few-shot prompting for the LLM, with both a SAFE and a DANGEROUS example, plus signal filtering at the data level (± 3) to keep noise out of the prompt.
- Edge-triggered packet alerts with cooldown — avoids notification spam while traffic stays high.
- Recent fix to use Cloudflare/Google DNS instead of the system resolver eliminated a class of silent false-negatives (e.g. SPF/DMARC reported as missing on Instagram) and made scans much faster.

Concerns and rough edges

Security issues that need fixing before any real distribution

- **Passwords are stored in plaintext** in the User table (src/SQLAlchemy/database.py). This needs to be hashed (bcrypt or argon2) before this app touches a real user.
- **A hardcoded user ('Nico', password 'asdf') is created on first run.** Useful during development; needs to come out before any kind of release.
- **The Flask backend has no authentication.** Any process on the machine can hit /add_to_blacklist, /scan, etc. A token (set when the desktop app starts, sent by the extension) would close this off.
- **Database engine uses echo=True**, which prints every SQL query to the console. Performance noise in production, and could leak data.

Code-quality items

- Many **bare 'except Exception: pass'** blocks throughout the codebase. They hide real bugs and make troubleshooting much harder than it needs to be.
- **No tests.** No test_*.py files anywhere — this is the single biggest risk to long-term maintenance. Even a small pytest suite covering the scoring rules and the cache logic would pay off.
- **print() used for logging.** Python's logging module gives you log levels, file output, and zero-overhead disabling — well worth the small refactor.
- **Magic numbers** scattered throughout: 12,000-packet threshold, 15s VT cooldown, the score thresholds for each verdict, and so on. A single config file (TOML or JSON) would centralize these.
- **Mixed path styles.** Some places use forward slashes, some use double-backslashes, some use pathlib. Standardizing on pathlib would help.
- **Multiple Python environments on the dev machine** (system 3.14, miniconda, Python 3.11). We hit this directly with the llama-cpp-python install. A venv pinned to one Python in the repo would prevent future churn.

What's missing or could be added

Detection improvements (the technical core)

- **Phishing / typosquatting detection.** Your current scoring catches structural badness (no HTTPS, no email config) but not lookalike domains. A character-level model — or even just Levenshtein distance against a list of well-known brands — would cover homoglyph attacks (paypa1.com, gooog1e.com, microsf1.com).
- **Threat intel feed integration.** Free public feeds (PhishTank, OpenPhish, abuse.ch URLhaus) could be pulled daily and checked locally before any heuristic scoring. Catches known-bad sites instantly with no API call.
- **Cache invalidation.** Right now a domain scanned once is cached forever. A 24-hour or 7-day TTL would keep results fresh without hammering the scanner.
- **ML-based scoring** (your professor's original AI request). Even without a labeled dataset, the existing extracted features (domain age, TTL, header counts, etc.) could feed a simple Random Forest trained on a public phishing corpus, giving you something measurable for the report.
- **DNS anomaly detection beyond thresholds.** The current packet alert is a single fixed number. An Isolation Forest or rolling Z-score on the per-second buckets you already collect would catch unusual patterns specific to the user's normal traffic.

UX / polish

- **Tooltips on signals.** Hovering over 'dns_mail: +6' in the additional info dialog could explain what MX/SPF/DMARC mean. Right now the user has to know.
- **Search and filter in history.** Once a user has hundreds of entries the current table will become hard to navigate.
- **Export to CSV/JSON.** Useful both for users and for instructors evaluating the project.
- **Printable scan report.** The data is there — a 'Save as PDF' button on a scan result would be a nice touch.
- **First-run onboarding.** Brief overlay explaining the four extension modes, since the Safe-mode flow is the most distinctive feature and easy to miss.

Distribution and ops

- **Real installer.** Inno Setup or NSIS could produce a one-click .exe. The Ollama / Llama-cpp model download you already built fits this pattern perfectly: ship the small app, fetch the 2 GB model on first launch.
- **Single-instance enforcement.** Right now nothing prevents two app instances racing for the same Flask port and database.
- **Auto-start option.** Optional 'Start with Windows' toggle in Settings.

- **Firefox extension.** Manifest V3 is broadly compatible — the existing extension would port with relatively small changes and double the addressable users.

Closing thought

For a capstone, this is a solid, ambitious project. It's not a toy — the components actually talk to each other, the heuristics produce useful signals, and the recent AI work shows good judgment about when to use a small local model versus a hosted API. The path from here to something genuinely shippable is mostly about hardening: hashed passwords, a backend auth token, tests for the scoring engine, and a config file. The detection improvements (typosquatting, threat-intel feeds, ML augmentation) are where the project can grow next, and any one of them would make a compelling chapter in a final report.